Kobuki Node

Thumrongsak Kosiyatrakul

Kobuki Node

- The `kobuki` node communicates with a Kobuki robot via USB
- Provide hardware abstractions
  - Motors with synchronization
  - Bumper switches
  - Buttons
  - LEDs
- To control a robot, simply communicate with the `kobuki` node
  - Publish commands to the `kobuki` node
  - Subscribe for information about the robot
- To start a kobuki node, use the following command:

```
roslaunch kobuki_node minimal.launch
```

- To ensure the stability of the robot's odometry, the robot should be on the ground/table before executing the above command

Kobuki's Subscriptions and Publications

- By default the `kobuki` node subscribes to the following topic:
  - `/mobile_base/commands/reset_odometry`
  - `/mobile_base/commands/sound`
  - `/mobile_base/commands/led1`
  - `/mobile_base/commands/led2`
  - `/mobile_base/commands/velocity`
- We simply publish data to the above topics to control the robot
- The `kobuki` node publishes to the following topic:
  - `/odom`
  - `/mobile_base/events/bumper`
  - `/mobile_base/events/button`
- We simply subscribe to the above topics to receive data from the robot
- There are more topics related to kobuki node but we will only focus on topics listed above.

Sounds

- A Kobuki robot has an ability to make sounds
- Simply publish a message of type `Sound` to the topic named

```
/mobile_base/commands/sound
```

- To recognize a message of type `Sound`, we need to import it

```
from kobuki_msgs.msg import Sound
```

- `Sound` message consists of only one component named `value`
- Simply set the value between 0 and 6 and publish
- Values:
  - 0 - turn on
  - 1 - turn off
  - 2 - recharge start
  - 3 - press button
  - 4 - error sound
  - 5 - start cleaning
  - 6 - cleaning end

Sounds

```
#!/usr/bin/env python

import rospy
from kobuki_msgs.msg import Sound

def sendSounds():
    rospy.init_node('sound_sender', anonymous=True)
    pub = rospy.Publisher('/mobile_base/commands/sound', Sound,
                          queue_size=10)
    while pub.get_num_connections() == 0:
        pass
    s = Sound()
    for x in range (0,7):
        s.value = x
        pub.publish(s)
        rospy.sleep(1.5)

if __name__ == '__main__':
    try:
        sendSounds()
    except rospy.ROSInterruptException:
        pass
```

LEDs

- There are two LEDs on a Kobuki robot named LED1 and LED2
- To control an LED, simply publish a message type `Led` to its associate topic:
  - LED1: `/mobile_base/commands/led1`
  - LED2: `/mobile_base/commands/led2`
- To recognize a message of type `Led`, we need to import it

```
from kobuki_msgs.msg import Led
```

- `Led` message consists of only one component named `value`
- Simply set the value and publish
- Values:
  - 0 - Black (Off)
  - 1 - Green
  - 2 - Orange
  - 3 - Red

LEDs

```
#!/usr/bin/env python

import rospy
from kobuki_msgs.msg import Led

def sendLEDs():
    rospy.init_node('leds_sender', anonymous=True)
    pub1 = rospy.Publisher('/mobile_base/commands/led1', Led, queue_size=10)
    pub2 = rospy.Publisher('/mobile_base/commands/led2', Led, queue_size=10)
    while pub1.get_num_connections() == 0 or pub2.get_num_connections() == 0:
        pass
    led = Led()
    x = 3
    while x != -1:
        led.value = x
        pub1.publish(led)
        x = x - 1
        rospy.sleep(1)

    x = 3
    while x != -1:
        led.value = x
        pub2.publish(led)
        x = x - 1
        rospy.sleep(1)

if __name__ == '__main__':
    try:
        sendLEDs()
    except rospy.ROSInterruptException:
        pass
```

Bumpers

- There are three bumpers on a Kobuki robot, left, center, and right
- If a bumper is pressed or release, it will publish a message of type `BumperEvent` to the topic:

```
/mobile_base/events/bumper
```

- Need to import the message of type `BumperEvent` as usual:

```
from kobuki_msgs.msg import BumperEvent
```

- `BumperEvent` consists of two components:
  - `bumper` with the following possible values:
    - 0 - Left
    - 1 - Center
    - 2 - Right
  - `state` with the following possible values:
    - 0 - Released
    - 1 - Pressed
- Simply subscribe to the topic to receive the message:

```
rospy.Subscriber('/mobile_base/events/bumper', BumperEvent, bumperCallback)
```

Bumpers

```
#!/usr/bin/env python

import rospy
from kobuki_msgs.msg import BumperEvent

def bumperCallback(data):
    str = ""
    if data.bumper == 0:
        str = str + "Left bumper is "
    elif data.bumper == 1:
        str = str + "Center bumper is "
    else:
        str = str + "Right bumper is "

    if data.state == 0:
        str = str + "released."
    else:
        str = str + "pressed."

    rospy.loginfo(str)

def bumperExample():
    rospy.init_node('bumper_example', anonymous=True)
    rospy.Subscriber('/mobile_base/events/bumper', BumperEvent, bumperCallback)
    rospy.spin()

if __name__ == '__main__':
    bumperExample()
```

Buttons

- Kobuki robot consists of three buttons, B0, B1, and B2
- If a button is pressed or released, a message of type `ButtonEvent` will be published into the topic

```
/mobile_base/events/button
```

- Your program must `import` the message type `ButtonEvent` as shown below:

```
from kobuki_msgs.msg import ButtonEvent
```

- The message of type `ButtonEvent` consists of two components:
  - `button` with the following possible values:
    - 0 - B0
    - 1 - B1
    - 2 - B2
  - `state` with the following possible values:
    - 0 - released
    - 1 - pressed
- Simply subscribe to the topic as usual:

```
rospy.Subscriber('/mobile_base/events/button', ButtonEvent, bumperCallback)
```

Button

```
#!/usr/bin/env python

import rospy
from kobuki_msgs.msg import ButtonEvent

def buttonCallback(data):
    str = ""
    if data.button == 0:
        str = str + "B0 is "
    elif data.button == 1:
        str = str + "B1 is "
    else:
        str = str + "B2 is "

    if data.state == 0:
        str = str + "released."
    else:
        str = str + "pressed."

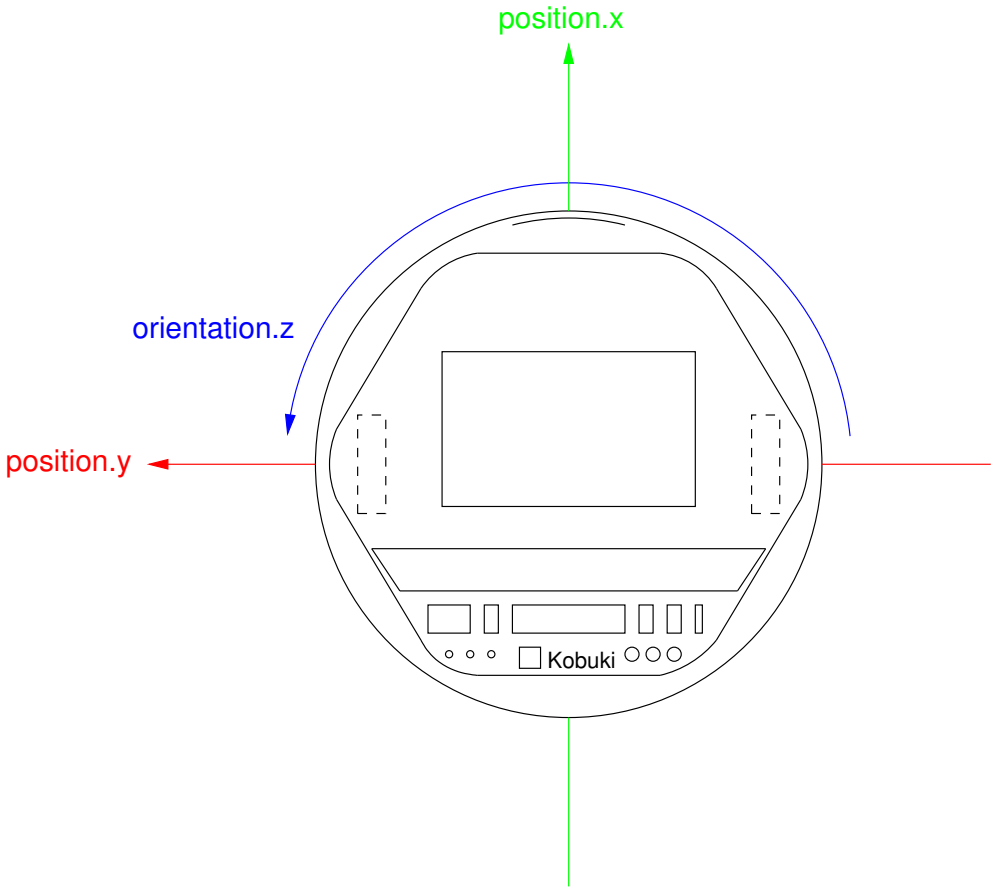
    rospy.loginfo(str)

def bumperExample():
    rospy.init_node('button_example', anonymous=True)
    rospy.Subscriber('/mobile_base/events/button', ButtonEvent, buttonCallback)
    rospy.spin()

if __name__ == '__main__':
    bumperExample()
```

Odometry

- Represents an estimate of a robot position in free space
- A Kobuki robot has three main information related to position and orientation:



- Information from an odometry allows us to move the robot with better precision

Odometry

- Type: `nav_mags/Odometry.msg`
- The component that we will focus on in `Odometry` message is `pose`
- The `pose` component contains two components and one of them is named `pose` (again)
- The inner `pose` component consists of two components:
  - `position` (in meter) consists of 3 components of type float named `x`, `y`, and `z`
  - `orientation` (in quaternion) consists of 4 components of type float named `x`, `y`, `z`, and `w`
- For a Kobuki robot:
  - `pose.pose.position.z` is always 0
  - `pose.pose.orientation.x` is always 0
  - `pose.pose.orientation.y` is always 0

Odometry

- Note that the orientation from Kobuki's odometry is in **quaternion**
- Quaternion is a unit suitable for rotating objects in 3D
- A quaternion value consists of four values, `x`, `y`, `z`, and `w`
- We generally familiar with roll, pitch, and yaw
- Since a Kobuki can only turn about the Z-axis, only yaw is applicable
- We can calculate yaw (**in radian**) using the following formula:

$$\texttt{yaw} = \arcsin(2 * x * y + 2 * z * w)$$

- Because x and y orientations of a Kobuki robot are always 0,

$$\texttt{yaw} = \arcsin(2 * z * w)$$

- To convert radian to degree simply multiply by $\frac{180}{\pi}$

Reset Kobuki's Odometry

- Often time, we may want to reset the odometry to 0
- This can be done by publishing an `Empty` message to

```
/mobile_base/commands/reset_odometry
```

- You must make sure that your program already establish a connection with the robot first
- Example:

```
#!/usr/bin/env python

import rospy
from std_msgs.msg import Empty

def resetter():
    pub = rospy.Publisher('/mobile_base/commands/reset_odometry',
                          Empty, queue_size=10)
    rospy.init_node('resetter', anonymous=True)
    while pub.get_num_connections() == 0:
        pass
    pub.publish(Empty())

if __name__ == '__main__':
    resetter()
```

Odometry

- Topic: `/odom`
- Purpose: Information about robot's where about
- Type: `nav_msgs/Odometry.msg`
- If you create a function named `odomCallback` that will be called every the program receive message from the topic `/odom` as follows:

```
def odomCallback(data):
```

  - `data.pose.pose.position.x` is the position of the robot from the origin along the x-axis
  - `data.pose.pose.position.y` is the position of the robot from the origin along the y-axis
  - `data.pose.pose.orientation.z` together with `data.pose.pose.orientation.w` is the direction where the robot is facing (quaternion).

odom

```
#!/usr/bin/env python

import rospy
import math
from nav_msgs.msg import Odometry
from tf.transformations import euler_from_quaternion

def odomCallback(data):
    # Convert quaternion to degree
    q = [data.pose.pose.orientation.x,
         data.pose.pose.orientation.y,
         data.pose.pose.orientation.z,
         data.pose.pose.orientation.w]
    roll, pitch, yaw = euler_from_quaternion(q)
    # roll, pitch, and yaw are in radian
    degree = yaw * 180 / math.pi
    x = data.pose.pose.position.x
    y = data.pose.pose.position.y
    msg = "(%.6f,%.6f) at %.6f degree." % (x, y, degree)
    rospy.loginfo(msg)

def odomExample():
    rospy.init_node('odom_example', anonymous=True)
    rospy.Subscriber('/odom', Odometry, odomCallback)
    rospy.spin()

if __name__ == '__main__':
    odomExample()
```